Technical Methodology Documentation

Deep Learning-Based Detection and Mapping of

Small-Scale Mining (Galamsey)

Centre for Remote Sensing and Geographic Information Services (CERSGIS)
University of Ghana, Legon

March 2026

Table of Contents

Table of Contents 2
1. Purpose and Scope 3
2. Data Requirements 3
  2.1 Satellite imagery 3
  2.2 Training labels 4
  2.3 Built-up mask 4
  2.4 Software environment 4
3. Methodology 4
  3.1 Stage 1: Mask creation 5
  3.2 Stage 2: Patch generation 6
  3.3 Stage 3: Spatial data splitting 6
  3.4 Stage 4: Model training 6
    3.4.1 Architecture 6
    3.4.2 Loss function 8
    3.4.3 Training configuration 8
    3.4.4 Data augmentation 10
    3.4.5 Model selection 10
  3.5 Stage 5: Evaluation 10
  3.6 Stage 6: Mosaic prediction 11
  3.7 Stage 7: Post-processing 12
    3.7.1 Nodata masking 12
    3.7.2 NDVI filter 13
    3.7.3 Built-up mask 13
    3.7.4 Small patch removal 13
4. References 14

1. Purpose and Scope

This report documents the methodology for a deep learning pipeline developed at CERSGIS to detect and map small-scale mining activities (galamsey) from Planet NICFI satellite imagery. The pipeline accepts a mosaiced satellite image and a set of manually digitised training polygons as inputs and produces a binary prediction map classifying each pixel as galamsey or non-galamsey.

The pipeline adapts established approaches in satellite-based artisanal mining detection. The overall detection strategy, comprising hand-labelled training data, neural network prediction, NDVI-based post-processing, and iterative refinement through successive review cycles, follows the framework developed by the Amazon Mining Watch project (earthrise-media/mining-detector, GitHub, MIT licensed). The pixel-level segmentation approach using a U-Net architecture draws from Couttenier et al. (2022), who demonstrated this method for artisanal mine mapping across 1.75 million square kilometres of Sub-Tropical West Africa. The specific model configuration, a U-Net with ResNet34 encoder pretrained on ImageNet applied to 256 × 256 pixel patches, follows the implementation pattern used in the LandCoverNet segmentation pipeline (pavlo-seimskyi/semantic-segmentation-satellite-imagery, GitHub) and across the broader satellite-image-deep-learning open-source ecosystem.

2. Data Requirements

2.1 Satellite imagery

The pipeline uses Planet NICFI analysis-ready mosaics: cloud-free composites at approximately 4.77-metre spatial resolution with four spectral bands (Blue, Green, Red, Near-Infrared). Surface reflectance values are stored as integers scaled by 10,000. The imagery is delivered as GeoTIFF tiles that must be mosaiced into a single file prior to processing.

Planet NICFI was selected for its spatial resolution. At 4.77 metres, individual mining pits and tailings ponds are distinguishable. Galamsey operations in Ghana tend to be smaller and more fragmented than the Amazonian operations targeted by Sentinel-2 based approaches, making the higher resolution operationally necessary.

2.2 Training labels

Training labels are prepared as a polygon shapefile with an integer field named *Class*. Two classes are used:

Class 1 (Galamsey): Polygons delineating mining footprints including active pits and degraded mining corridors.

Class 0 (Non-galamsey / Hard negatives): Polygons delineating spectrally similar non-mining features: bare farmland, quarries, gravel pits, road construction, settlements, and natural bare areas. These train the model to discriminate between mining activity and other exposed surfaces that share similar spectral characteristics.

Exhaustive labelling of the full study area is not required. Unlabelled pixels receive a mask value of 255 and are excluded from the loss function during training. Only explicitly annotated pixels contribute to parameter updates.

2.3 Built-up mask

A binary raster identifying settlements, roads, and built-up areas is used during post-processing. Suitable sources include the JRC Global Human Settlement Layer, OpenStreetMap, or institutional datasets, resampled to match the mosaic.

2.4 Software environment

The pipeline is implemented in Python 3.11 using PyTorch. Model training uses Google Colab (NVIDIA T4 GPU). Data preparation and post-processing run locally on CPU. The segmentation-models-pytorch library (Yakubovskiy, 2019) provides the model architecture. Additional dependencies: *rasterio*, *geopandas*, *albumentations*, *scikit-learn*, *scipy*.

3. Methodology

The pipeline consists of seven stages: mask creation, patch generation, spatial data splitting, model training, evaluation, mosaic prediction, and post-processing. Figure 1 provides an overview of the complete pipeline.

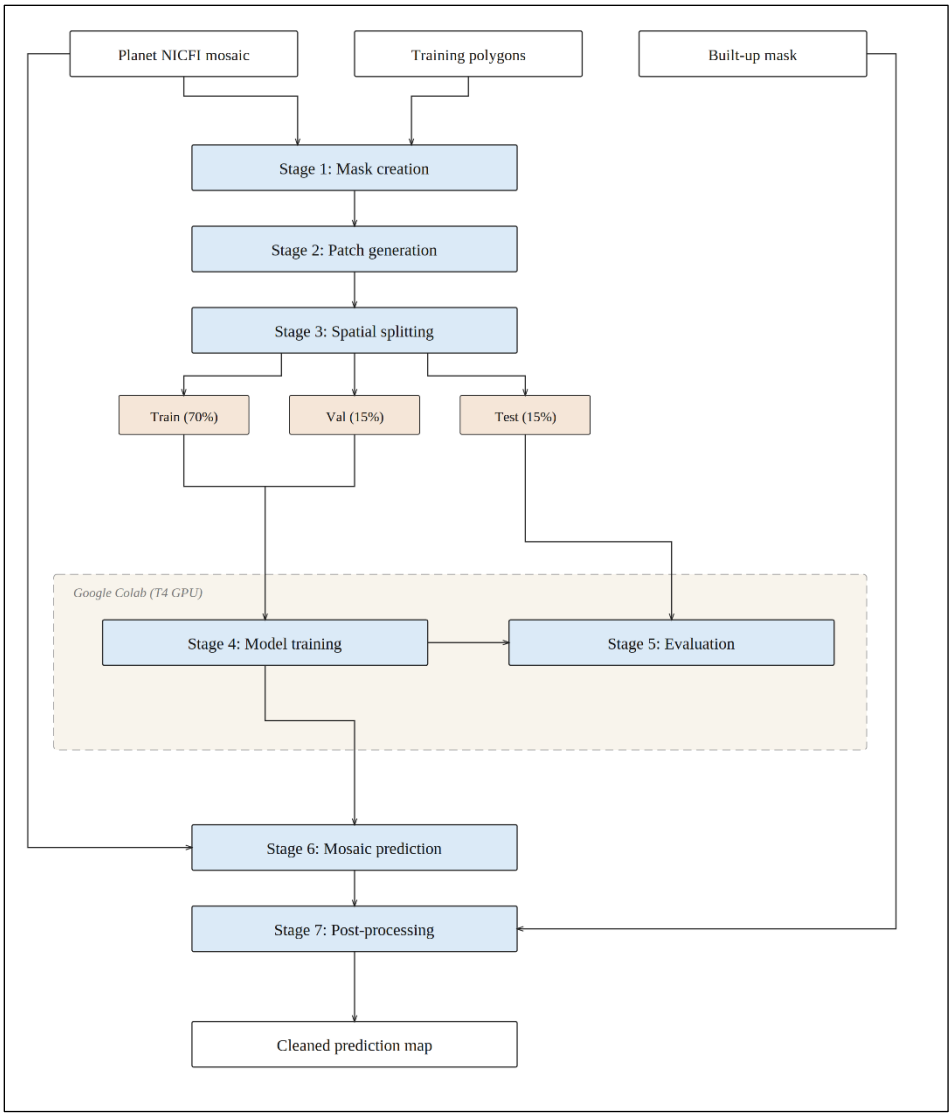

Figure 1. *Pipeline overview. Stages 1–3 run locally; Stages 4–5 run on Google Colab (T4 GPU); Stages 6–7 run locally.*

3.1 Stage 1: Mask creation

The digitised polygons are rasterised into a single-band mask aligned pixel-for-pixel with the mosaic, encoding three states: 1 (galamsey), 0 (non-galamsey), and 255 (unlabelled).

1. The mosaic is read to extract pixel dimensions, geospatial transform, and CRS.
2. The shapefile is reprojected to match the mosaic CRS if necessary.
3. A blank raster is initialised with all pixels set to 255.
4. Class 0 polygons are rasterised first, followed by Class 1. Where polygons overlap, Class 1 values overwrite Class 0, ensuring that mining labels take precedence.

5. The mask is saved as a single-band GeoTIFF.

3.2 Stage 2: Patch generation

The mosaic and mask are sliced into paired 256 × 256 pixel patches with 64-pixel overlap. At 4.77 m/pixel, each patch covers approximately 1.22 × 1.22 km on the ground. The overlap ensures boundary features are fully represented and enables overlap-averaging during prediction. A minimum labelled pixel filter discards patches where fewer than 5% of pixels carry a class label.

3.3 Stage 3: Spatial data splitting

Patches are divided into training (70%), validation (15%), and test (15%) sets using spatially-aware block splitting. Patches are grouped into contiguous blocks of 1,024 pixels (approximately 4.9 km), and each block is assigned in its entirety to a single set. This prevents spatial autocorrelation from inflating performance metrics, following the spatial separation principle applied by Couttenier et al. (2022).

3.4 Stage 4: Model training

3.4.1 Architecture

The model is a U-Net (Ronneberger et al., 2015) with a ResNet34 encoder (He et al., 2016) pretrained on ImageNet. The U-Net encoder-decoder structure with skip connections combines high-level semantic understanding with fine spatial detail. ResNet34 provides sufficient representational capacity for the scale of training data used in this pipeline while maintaining manageable computational requirements. ImageNet pretraining provides robust initial feature representations, reducing the volume of task-specific training data required. Figure 3 illustrates the architecture.

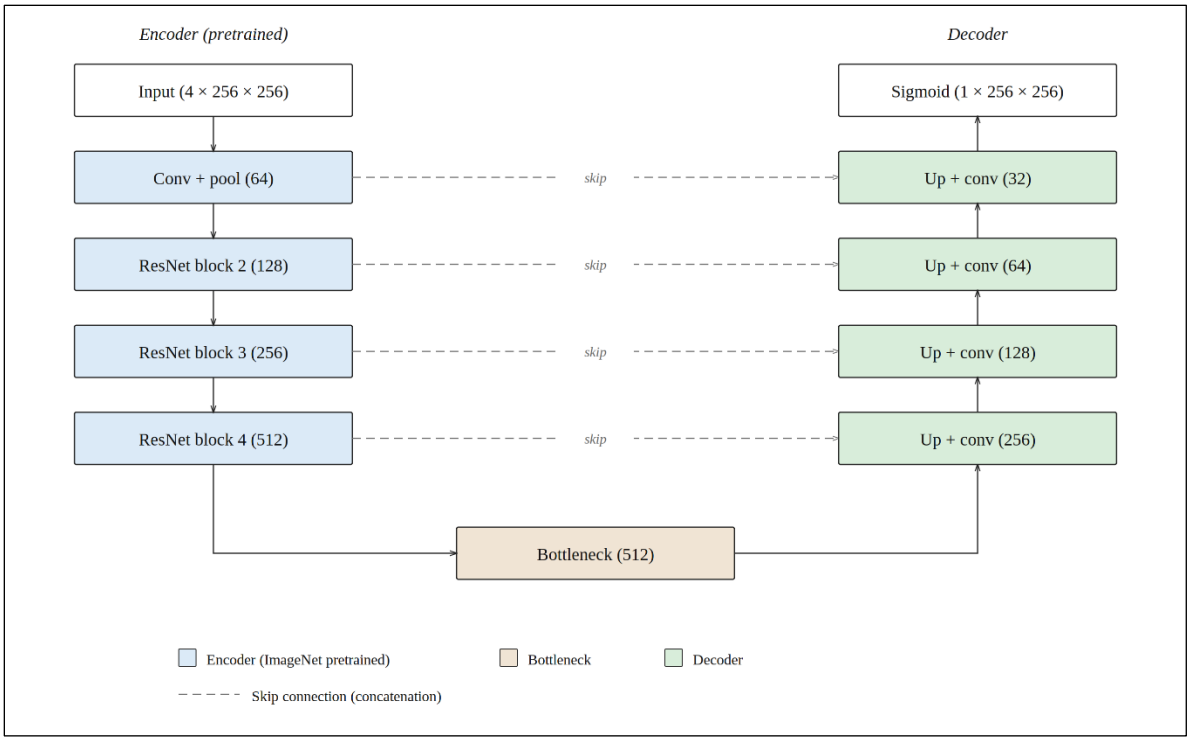

Figure 2. *U-Net architecture. The encoder (ResNet34, ImageNet pretrained) compresses the input through successive blocks while the decoder reconstructs spatial resolution. Skip connections concatenate encoder features with decoder features at each level.*

The model accepts 4 input channels corresponding to the four Planet NICFI spectral bands (Blue, Green, Red, Near-Infrared) and produces a single output channel representing the per-pixel probability of galamsey.

```
# Model
model = smp.Unet(
    encoder_name=ENCODER,
    encoder_weights='imagenet',
    in_channels=IN_CHANNELS,
    classes=1,
).to(device)
```

Figure 3. *Model instantiation. ENCODER and IN_CHANNELS are configuration variables set to 'resnet34' and 4 respectively.*

3.4.2 Loss function

Training uses a combined Binary Cross-Entropy (BCE) and Dice loss function. BCE optimises per-pixel classification accuracy, while Dice loss maximises the spatial overlap between predicted and reference regions, penalising fragmented predictions. Pixels with a mask value of 255 are excluded from the loss computation via a validity mask.

```python
# Loss Function
class CombinedLoss(nn.Module):
    def __init__(self):
        super().__init__()
        self.bce = nn.BCEWithLogitsLoss(reduction='none')

    def forward(self, pred, target):
        pred = pred.squeeze(1)
        target = target.long()
        valid = (target != 255).float()
        if valid.sum() == 0:
            return torch.tensor(0.0, device=pred.device, requires_grad=True)
        target_float = target.float().clamp(0, 1)
        bce_loss = (self.bce(pred, target_float) * valid).sum() / valid.sum()
        pred_sig = torch.sigmoid(pred)
        pred_valid = pred_sig * valid
        target_valid = target_float * valid
        intersection = (pred_valid * target_valid).sum()
        union = pred_valid.sum() + target_valid.sum()
        dice_loss = 1 - (2 * intersection + 1) / (union + 1)
        return bce_loss + dice_loss

criterion = CombinedLoss()
```

Figure 4. *Combined BCE-Dice loss function. Only labelled pixels (0 or 1) contribute to the loss computation. Unlabelled pixels (255) are excluded via a validity mask.*

3.4.3 Training configuration

Parameter	Value	Rationale
Optimiser	Adam	Adaptive learning rate optimiser (Kingma and Ba, 2015). Widely adopted for fine-tuning pretrained convolutional networks due to per-parameter learning rate adjustment.
Learning rate	0.0001	Conservatively low rate to preserve pretrained encoder weights while allowing task-specific adaptation. Higher rates risk destabilising learned feature representations.

LR schedule	ReduceLROnPlateau	Halves the learning rate after 5 epochs without validation improvement. Permits coarse parameter updates in early training and progressively finer adjustments as the model converges.
Epochs	50	Empirically sufficient for convergence on this dataset. The best-performing model weights are saved based on validation loss, preventing overfitting beyond the optimal epoch.
Batch size	16	Maximum batch size that fits within T4 GPU memory (16 GB) for 4-channel 256×256 input patches.
Normalisation	÷10,000	Rescales Planet NICFI surface reflectance integers (stored with a scale factor of 10,000) to a 0–1 floating-point range for model input.

```python
# Training Loop
best_val_loss = float('inf')
train_losses, val_losses = [], []

print(f'{"Epoch":>6} {"Train Loss":>12} {"Val Loss":>12} {"LR":>10} {"Status":>10}')
print('-' * 55)

for epoch in range(NUM_EPOCHS):
    model.train()
    running = 0.0
    for imgs, msks in train_loader:
        imgs, msks = imgs.to(device), msks.to(device)
        optimizer.zero_grad()
        loss = criterion(model(imgs), msks)
        loss.backward()
        optimizer.step()
        running += loss.item()
    train_loss = running / len(train_loader)
    train_losses.append(train_loss)

    model.eval()
    running_val = 0.0
    with torch.no_grad():
        for imgs, msks in val_loader:
            imgs, msks = imgs.to(device), msks.to(device)
            running_val += criterion(model(imgs), msks).item()
    val_loss = running_val / len(val_loader)
    val_losses.append(val_loss)

    scheduler.step(val_loss)
    lr = optimizer.param_groups[0]['lr']

    status = ''
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), '/content/best_model.pth')
        status = 'saved'

    print(f'{epoch+1:>6} {train_loss:>12.4f} {val_loss:>12.4f} {lr:>10.6f} {status:>10}')

print(f'\nBest val loss: {best_val_loss:.4f}')
```

Figure 5. *Training loop. The model trains for 50 epochs with early stopping based on validation loss. The best model weights are saved when validation loss improves.*

3.4.4 Data augmentation

Training patches are augmented at runtime using the albumentations library to increase the effective training set size and reduce overfitting.

Transformation	Probability	Rationale
Horizontal flip	0.5	Mining features exhibit no preferred lateral orientation in satellite imagery.
Vertical flip	0.5	Mining features exhibit no preferred vertical orientation in satellite imagery.
Random 90° rotation	0.5	Mining features exhibit no preferred compass orientation in nadir satellite imagery.
Brightness/contrast	0.3	Simulates variation in atmospheric conditions and solar illumination across acquisitions.

3.4.5 Model selection

Model weights corresponding to the lowest validation loss across all epochs are saved as the final model.

```
# Save Best Model
shutil.copy2('/content/best_model.pth', os.path.join(DRIVE_DIR, 'best_model.pth'))
print(f'Model saved to: {DRIVE_DIR}/best_model.pth')
```

Figure 6. *Model checkpoint corresponding to the lowest validation loss is saved for inference.*

3.5 Stage 5: Evaluation

The model is evaluated on the held-out test set by thresholding predicted probabilities at 0.5. Only labelled pixels contribute to the evaluation; pixels with a mask value of 255 are excluded. Four standard segmentation metrics are computed: Precision, Recall, F1 Score, and Intersection over Union (IoU).

```python
# Model Evaluation
model.load_state_dict(torch.load('/content/best_model.pth', weights_only=True))
model.eval()

all_preds, all_targets = [], []
with torch.no_grad():
    for imgs, msks in test_loader:
        preds = (torch.sigmoid(model(imgs.to(device)).squeeze(1)) > 0.5).cpu().numpy()
        msks = msks.numpy()
        for p, m in zip(preds, msks):
            valid = m != 255
            all_preds.append(p[valid])
            all_targets.append(m[valid])

all_preds = np.concatenate(all_preds)
all_targets = np.concatenate(all_targets)

tp = ((all_preds == 1) & (all_targets == 1)).sum()
fp = ((all_preds == 1) & (all_targets == 0)).sum()
fn = ((all_preds == 0) & (all_targets == 1)).sum()
tn = ((all_preds == 0) & (all_targets == 0)).sum()

precision = tp / (tp + fp + 1e-8)
recall = tp / (tp + fn + 1e-8)
f1 = 2 * precision * recall / (precision + recall + 1e-8)
iou = tp / (tp + fp + fn + 1e-8)

print('Test Set Evaluation')
print('=' * 40)
print(f'Precision: {precision:.4f}')
print(f'Recall:    {recall:.4f}')
print(f'F1 Score:  {f1:.4f}')
print(f'IoU:       {iou:.4f}')
print(f'\nTP: {tp:,}  FP: {fp:,}  FN: {fn:,}  TN: {tn:,}')
```

Figure 7. *Evaluation on the held-out test set. Metrics are computed on labelled pixels only; 255-masked pixels are excluded.*

3.6 Stage 6: Mosaic prediction

The trained model is applied to the full mosaic using a sliding window approach within tiled blocks. The mosaic is divided into large blocks (10,000 pixels per side) with 256-pixel overlap buffers to eliminate edge artefacts between blocks. Within each block, a 256 × 256 pixel sliding window with 64-pixel stride passes over the imagery. At each position, bands are normalised to 0–1 and passed through the model. Where patches overlap, predictions are averaged to produce smoother boundaries. The result is thresholded at 0.5 to produce a binary classification. The output is a single-band GeoTIFF matching the mosaic CRS, extent, and resolution.

```python
# Footprint prediction
def predict_block(src, model, y_start, x_start, block_h, block_w, buffer):
    """Predict a single block with buffer overlap. Returns clipped binary result."""
    buf_y = max(0, y_start - buffer)
    buf_x = max(0, x_start - buffer)
    buf_y_end = min(FULL_HEIGHT, y_start + block_h + buffer)
    buf_x_end = min(FULL_WIDTH, x_start + block_w + buffer)
    buf_h = buf_y_end - buf_y
    buf_w = buf_x_end - buf_x

    stride = PATCH_SIZE - OVERLAP
    pred_sum = np.zeros((buf_h, buf_w), dtype=np.float32)
    pred_count = np.zeros((buf_h, buf_w), dtype=np.float32)

    with torch.no_grad():
        for py in range(0, buf_h - PATCH_SIZE + 1, stride):
            for px in range(0, buf_w - PATCH_SIZE + 1, stride):
                window = Window(buf_x + px, buf_y + py, PATCH_SIZE, PATCH_SIZE)
                img = src.read(window=window).astype(np.float32)
                img = np.nan_to_num(img, nan=0.0, posinf=0.0, neginf=0.0)
                img = np.clip(img / 10000.0, 0, 1)  # Planet NICFI reflectance to 0-1

                tensor = torch.tensor(img).unsqueeze(0).to(device)
                pred = torch.sigmoid(model(tensor)).squeeze().cpu().numpy()

                pred_sum[py:py+PATCH_SIZE, px:px+PATCH_SIZE] += pred
                pred_count[py:py+PATCH_SIZE, px:px+PATCH_SIZE] += 1

    pred_count[pred_count == 0] = 1
    pred_avg = pred_sum / pred_count

    # Trim the buffer to get just the target block
    clip_y = y_start - buf_y
    clip_x = x_start - buf_x
    result = pred_avg[clip_y:clip_y+block_h, clip_x:clip_x+block_w]

    return (result > 0.5).astype(np.uint8)
```

Figure 8. *Mosaic prediction. Sliding window with overlap averaging produces a continuous probability map, thresholded at 0.5 to produce a binary prediction.*

3.7 Stage 7: Post-processing

The raw binary prediction is refined through a sequence of rule-based filters, each targeting a specific category of false positive. All post-processing steps operate on strips of the raster to remain within memory constraints on large mosaics.

3.7.1 Nodata masking

Pixels where all four mosaic bands equal zero indicate areas outside the image footprint (tile edges, gaps between swaths). These pixels are set to zero in the prediction to prevent false positives along mosaic boundaries.

3.7.2 NDVI filter

Pixels with NDVI above a computed threshold are reclassified as non-galamsey. The threshold is determined using Otsu's method (Otsu, 1979), which analyses the NDVI distribution of predicted galamsey pixels and identifies the value that maximises between-class variance, separating bare ground from vegetation. This makes the threshold adaptive to the spectral conditions of each image without manual calibration.

3.7.3 Built-up mask

Predicted galamsey pixels coinciding with the built-up mask are reclassified as non-galamsey. This addresses spectral confusion between bare mining surfaces and exposed urban ground such as unpaved roads, construction sites, and cleared settlement areas. The built-up mask is derived from the JRC Global Human Settlement Layer and OpenStreetMap building footprints, resampled to match the mosaic resolution using nearest-neighbour interpolation.

3.7.4 Small patch removal

Isolated clusters smaller than 50 pixels (approximately 1,137 m^2) are removed using the GDAL sieve filter with 4-connected neighbourhood. This eliminates scattered noise pixels while preserving the boundaries of legitimate mining footprints.

4. References

Couttenier, M., Di Rollo, S., Inguere, L., Mohand, M. and Schmidt, L. (2022). Mapping artisanal and small-scale mines at large scale from space with deep learning. PLOS ONE, 17(9), e0267963. https://doi.org/10.1371/journal.pone.0267963

earthrise-media (2024). mining-detector: Detection of artisanal gold mines in satellite imagery for Amazon Mining Watch. GitHub. https://github.com/earthrise-media/mining-detector [MIT License]

He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE CVPR, pp. 770-778. https://arxiv.org/abs/1512.03385

Kingma, D.P. and Ba, J. (2015). Adam: A method for stochastic optimisation. In Proceedings of ICLR. https://arxiv.org/abs/1412.6980

Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), pp. 62–66. https://doi.org/10.1109/TSMC.1979.4310076

pavlo-seimskyi (2021). semantic-segmentation-satellite-imagery: U-Net with ResNet34 for land cover segmentation on LandCoverNet Africa. GitHub. https://github.com/pavlo-seimskyi/semantic-segmentation-satellite-imagery

Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In MICCAI, pp. 234-241. Springer. https://doi.org/10.1007/978-3-319-24574-4_28

satellite-image-deep-learning (2024). techniques: Techniques for deep learning with satellite and aerial imagery. GitHub. https://github.com/satellite-image-deep-learning/techniques

Yakubovskiy, P. (2019). segmentation-models-pytorch. GitHub. https://github.com/qubvel/segmentation_models.pytorch